

WikiPulse Technical Report: Real-Time Wikipedia Intelligence

1. Executive Summary

WikiPulse is a real-time streaming analytics platform designed to detect emerging global events by monitoring live Wikipedia edit activity. By leveraging **Wikimedia EventStreams**, the system captures the world's reactions to breaking news, political shifts, and cultural trends with sub-second latency. The project implements a decoupled, event-driven architecture using Python, Google Cloud Pub/Sub, and MongoDB Atlas.

2. System Architecture

The platform is built on a four-layer modular design to ensure high throughput and fault tolerance.

- **Ingestion Layer:** A resilient Python producer that maintains a persistent SSE (Server-Sent Events) connection to Wikimedia. It performs initial JSON validation and filters for edit and new event types before publishing to the cloud.
- **Messaging Layer (Decoupling):** Google Cloud Pub/Sub acts as the system's "shock absorber." It decouples the ingestor from the processor, providing a durable buffer that handles traffic spikes and prevents data loss during downstream maintenance.
- **Stream Processing Layer:** A Python consumer that transforms raw events into a standardized schema. It handles "noise" by filtering bot activity and ensures data integrity through idempotent writes (Upserts).
- **Storage & Analytics Layer:** Processed data is stored in MongoDB Atlas. Advanced aggregation pipelines analyze the collection to generate real-time insights, such as trending topics and editor distribution.

3. Design Decisions & Trade-offs

3.1 Streaming vs. Batch Processing

A streaming model was chosen because global events are time-sensitive. Batch processing would introduce "knowledge gaps" where critical signals are delayed. The trade-off is increased system complexity, requiring constant connection management and asynchronous error handling.

3.2 Schema Choice: Event-Based (Flat) Documents

We chose an **Event-Based Schema** over a Grouped (per-page) Schema.

- **Pros:** High write performance and historical accuracy. It avoids "write-lock" contention when thousands of people edit the same trending page simultaneously.
- **Cons:** Requires more storage space than summarized records, but the analytical flexibility outweighs the storage cost.

3.3 Messaging Layer: Cloud Pub/Sub

Pub/Sub was selected for its native **Flow Control** capabilities. This allows the system to manage "backpressure"—slowing down the ingestion rate if the database becomes overwhelmed—ensuring system stability without manual intervention.

4. Challenges & Solutions

- **Persistent Connections:** Wikimedia streams frequently drop due to network jitter.
 - *Solution:* Implemented a **Resilience Loop** with exponential backoff in the ingestion service to ensure 24/7 uptime.
- **Data Deduplication:** In distributed systems, messages can occasionally be delivered twice.
 - *Solution:* Implemented a **Unique Index** on the event_id in MongoDB. Using update_one with upsert=True ensures that even if an event is processed twice, only one record exists in storage.
- **Malformed Events:** Raw stream data occasionally contains empty "heartbeats" or broken JSON.
 - *Solution:* A dedicated **Initial Validation** step in the ingestor drops invalid payloads before they consume cloud credits or processing power.

5. Scalability Approach

The system is designed for **Horizontal Scaling**. Because the layers are decoupled via Pub/Sub:

1. If edit volume increases, we can deploy multiple instances of the processing.py script.
2. Google Cloud Pub/Sub automatically balances the load among all active consumers.
3. MongoDB Atlas can scale its clusters to handle increased write IOPS without changing a single line of code.

6. Conclusion

WikiPulse successfully demonstrates a modern data engineering pipeline. By prioritizing decoupling and idempotent processing, the platform provides a reliable foundation for real-time global event detection, transforming a noisy stream of edits into actionable intelligence.